

MODULE

2

Working with Text, Numbers, and Conditional Logic

Master every text manipulation function, number rounding and arithmetic, and the full power of conditional logic in M — building blocks for professional-grade data cleaning and transformation pipelines.

Module Contents

■ Module Overview

■ Lesson 1: Working with Text Functions

- Why Text Matters in Power Query
- Case Transformation: `Text.Upper`, `Text.Lower`, `Text.Proper`
- Whitespace: `Text.Trim`, `Text.TrimStart`, `Text.TrimEnd`
- Padding: `Text.PadStart`, `Text.PadEnd`
- Measuring and Inspecting: `Text.Length`, `Text.Contains`, `Text.StartsWith`, `Text.EndsWith`
- Extracting Substrings: `Text.Start`, `Text.End`, `Text.Middle`, `Text.Range`
- Searching: `Text.PositionOf`, `Text.PositionOfAny`
- Modifying: `Text.Replace`, `Text.ReplaceRange`, `Text.Remove`, `Text.Select`
- Splitting: `Text.Split`, `Text.SplitAny`
- Combining: `Text.Combine`, `Text.Insert`, `Text.Repeat`
- Converting: `Text.From`, `Text.ToList`, `Text.Format`

■ Lesson 2: Working with Number Functions

- Numbers in M: type number vs `Int64.Type` vs `Currency.Type`
- Rounding: `Number.Round`, `Number.RoundUp`, `Number.RoundDown`, `Number.RoundTowardZero`
- Arithmetic: `Number.Abs`, `Number.Sign`, `Number.Mod`, `Number.IntegerDivide`
- Power and Roots: `Number.Power`, `Number.Sqrt`, `Number.Exp`, `Number.Log`, `Number.Log10`
- Trigonometry: `Number.Sin`, `Number.Cos`, `Number.Tan` and inverses
- Constants and Limits: `Number.E`, `Number.PI`, `Number.Epsilon`, `Number.NaN`
- Checking Values: `Number.IsNaN`, `Number.IsEven`, `Number.IsOdd`
- Conversion: `Number.From`, `Number.ToText`

■ Lesson 3: Conditional Logic in M

- The if-then-else Expression
- Nested Conditions
- The each Keyword in Conditional Context
- Combining Conditions: and, or, not
- The Conditional Column Pattern
- Null-safe Conditionals
- Short-circuit Evaluation in Conditions

■ Lesson 4: The each Keyword and Row-Level Operations

- What each Really Means
- The Underscore `_` as Context Variable
- each in `Table.AddColumn` vs `Table.SelectRows`
- Nested each Expressions

- When NOT to Use each

■ Lesson 5: Combining Text, Numbers, and Logic in Real Queries

- Chaining Transformations with Table.TransformColumns
- Building Derived Columns with Table.AddColumn
- Real-world Pattern: Product Name Standardisation
- Real-world Pattern: Revenue Classification Pipeline
- Real-world Pattern: Address Parsing

■ Demonstration 1: Building a Name and Address Cleanup Pipeline

■ Demonstration 2: Creating a Multi-Tier Conditional Classification Column

■ Lab 2: Text, Numbers, and Conditional Logic

- Exercise 1: Text Transformations on Product and Customer Data
- Exercise 2: Number Calculations — Margins, Rounding, and Derivations
- Exercise 3: Conditional Logic — Classify, Flag, and Tier
- Exercise 4: Full Pipeline — Clean, Compute, Classify, and Load

■ Module Review and Takeaways

- Review Questions
- Best Practices
- Common Issues and Solutions
- Key Functions Quick Reference

Module Overview

In Module 1 you learned the structure of an M query, how the type system works, and the core table-manipulation functions. Module 2 goes deeper into the richest areas of everyday M work: **text processing**, **numeric computation**, and **conditional logic**.

Real-world data is messy. Product names arrive with inconsistent casing and trailing spaces. Order amounts need rounding to two decimal places. Sales tiers must be derived from raw revenue figures. Every one of these tasks maps to a specific group of M functions — and mastering them is what separates a proficient Power Query developer from someone who simply records clicks in the UI.

By the end of this module you will be able to write a complete data-cleaning pipeline entirely in M, handling text normalisation, numeric derivations, and multi-branch conditional columns — without touching the ribbon.

NOTE Module 2 builds directly on Module 1. You should be comfortable with the `let...in` structure, `Table.AddColumn`, `Table.SelectRows`, and `Table.TransformColumnTypes` before proceeding.

Before You Begin

- Complete Module 1 and its lab exercises.
- Have Power BI Desktop open with the Superstore.csv loaded (from Lab 1).
- Download the supplementary lab file: Contoso_Products.csv — available from the instructor or the course companion materials repository.
- Review the Applied Steps pane of your Lab 1 FactSales query — you will extend it.

LESSON 1

Working with Text Functions

After completing this lesson, you will be able to:

- Apply case transformation functions to standardise text columns.
- Remove leading, trailing, and internal whitespace with Trim functions.
- Measure text length and search for substrings using `Text.Length` and `Text.Contains`.
- Extract substrings using `Text.Start`, `Text.End`, `Text.Middle`, and `Text.Range`.
- Search for character positions using `Text.PositionOf`.

- Modify text using `Text.Replace`, `Text.Remove`, and `Text.Select`.
- Split text into lists using `Text.Split`, and combine text with `Text.Combine`.
- Convert values to text and format strings using `Text.From` and `Text.Format`.

Why Text Matters in Power Query

Text transformation is the most frequently performed operation in real-world data preparation. Source systems deliver data with inconsistent casing, embedded whitespace, concatenated fields, and mixed formats. Before any numeric analysis can happen, this text must be standardised. M provides a comprehensive library of text functions — all prefixed with **Text**. — that cover every common string manipulation scenario.

NOTE All M text functions work on Unicode strings. M's text type is UTF-16 internally. Text comparisons in M are case-sensitive and culture-aware by default. Functions that compare text accept an optional `Comparer` argument to control case sensitivity and culture: `Comparer.OrdinalIgnoreCase`, `Comparer.FromCulture`.

Case Transformation

Three functions handle case conversion. They are among the most-used text functions in any data cleaning pipeline:

Text.Upper

Syntax	<code>Text.Upper(text as nullable text, optional culture as nullable text) as nullable text</code>	
Parameter	Type	Description
<code>text</code>	<i>nullable text</i>	The input string. If null, returns null.
<code>culture</code>	<i>nullable text</i>	Optional. Culture code e.g. "en-US", "tr-TR". Affects locale-specific casing rules.
Returns	A new text value with all characters converted to uppercase.	
Example	<code>Text.Upper("hello world")</code>	
Result	"HELLO WORLD"	
NOTE	Culture matters for languages with special casing rules. In Turkish (tr-TR), the lowercase 'i' uppercases to 'İ' with a dot (I■), not plain 'I'. Always pass a culture for production data if locale consistency is required.	

Text.Lower

Syntax	Text.Lower(text as nullable text, optional culture as nullable text) as nullable text	
Parameter	Type	Description
text	nullable text	The input string.
culture	nullable text	Optional culture code.
Returns	A new text value with all characters converted to lowercase.	
Example	Text.Lower("CONTOSO Sales Report")	
Result	"contoso sales report"	

Text.Proper

Syntax	Text.Proper(text as nullable text, optional culture as nullable text) as nullable text	
Parameter	Type	Description
text	nullable text	The input string.
culture	nullable text	Optional culture code.
Returns	A new text value with the first letter of each word capitalised and all others lowercased.	
Example	Text.Proper("john paul jones")	
Result	"John Paul Jones"	
NOTE	Text.Proper capitalises every word — words are delimited by spaces, hyphens, and other non-letter characters. Be aware that abbreviations like 'USA' or 'LLC' will become 'Usa' and 'Llc'. Post-process with Text.Replace if needed.	

Whitespace Removal

Invisible whitespace characters are one of the leading causes of lookup failures and duplicate keys in data models. A Customer ID of 'CUST001 ' (trailing space) will not match 'CUST001' in a relationship. Three functions address whitespace:

Text.Trim

Syntax	Text.Trim(text as nullable text, optional char as any) as nullable text	
Parameter	Type	Description
text	nullable text	The input string.

<code>char</code>	<i>text or list</i>	Optional. Specific character(s) to trim. Default: all whitespace characters.
Returns	Removes all leading and trailing whitespace (or specified characters) from the text.	
Example	<code>Text.Trim(" hello world ")</code>	
Result	"hello world"	
NOTE	Text.Trim also accepts a list of characters to remove: <code>Text.Trim("***hello***", "***")</code> returns "hello". The default trims any whitespace character including tabs, non-breaking spaces (u00A0), and carriage returns.	
TIP	Text.Trim only removes characters from the start and end of the string. To collapse internal multiple spaces into one, use: <code>Text.Combine(Text.SplitAny(Text.Trim(value, " "), " "), " ")</code> or use a Text.Replace loop. Internal whitespace normalisation is covered in Lesson 5.	

Text.TrimStart	Removes leading whitespace (left side) only. Trailing whitespace is preserved.
<code>Text.TrimStart(text, optional char)</code>	<code>Text.TrimStart(" hello ")</code> → "hello "
Text.TrimEnd	Removes trailing whitespace (right side) only. Leading whitespace is preserved.
<code>Text.TrimEnd(text, optional char)</code>	<code>Text.TrimEnd(" hello ")</code> → " hello"

Padding: Text.PadStart and Text.PadEnd

Padding functions ensure a text value reaches a minimum length by adding a specified character. The most common use case is zero-padding numeric codes like order numbers or postal codes:

Text.PadStart

Syntax	<code>Text.PadStart(text as nullable text, count as number, optional char as nullable text) as nullable text</code>	
Parameter	Type	Description
<code>text</code>	<i>nullable text</i>	The input string to pad.
<code>count</code>	<i>number</i>	The desired total length of the result.
<code>char</code>	<i>nullable text</i>	Optional. The padding character. Default: space (" ").
Returns	Pads the beginning of the text with char until the total length equals count. If the text is already longer than count, it is returned unchanged.	
Example	<code>Text.PadStart("42", 6, "0")</code>	
Result	"000042"	

NOTE Use `Text.PadStart` for left-padding (e.g. product codes, zip codes). Use `Text.PadEnd` for right-padding (e.g. formatting fixed-width reports).

Text.PadEnd	Pads the end of text with char until the total length equals count.
<code>Text.PadEnd(text, count, optional char)</code>	<code>Text.PadEnd("ABC", 6, "-")</code> → "ABC---"

Measuring and Inspecting Text

Text.Length

Syntax	<code>Text.Length(text as nullable text) as nullable number</code>	
Parameter	Type	Description
<code>text</code>	<i>nullable text</i>	The input string. Returns null if text is null.
Returns	The number of characters in the text. Returns null if input is null.	
Example	<code>Text.Length("Power Query")</code>	
Result	11	
NOTE	Use <code>Text.Length</code> to detect abnormally short or long values (data quality checks), to calculate padding requirements, or to validate fixed-length codes.	

Text.Contains

Syntax	<code>Text.Contains(text as nullable text, substring as text, optional comparer as nullable function) as nullable logical</code>	
Parameter	Type	Description
<code>text</code>	<i>nullable text</i>	The string to search within.
<code>substring</code>	<i>text</i>	The string to search for.
<code>comparer</code>	<i>nullable function</i>	Optional. Use <code>Comparer.OrdinalIgnoreCase</code> for case-insensitive search.
Returns	True if substring is found anywhere in text; false otherwise. Returns null if text is null.	
Example	<code>Text.Contains("Power Query M", "query", Comparer.OrdinalIgnoreCase)</code>	
Result	true	
NOTE	Without a comparer, <code>Text.Contains</code> is case-sensitive: <code>Text.Contains("Hello", "hello")</code> returns false. Always pass <code>Comparer.OrdinalIgnoreCase</code> when case should not matter.	

Text.StartsWith	Returns true if text begins with the specified substring.
Text.StartsWith(text, substring, optional comparer)	Text.StartsWith("PROD-001", "PROD") → true

Text.EndsWith	Returns true if text ends with the specified substring. Useful for file extension checks.
Text.EndsWith(text, substring, optional comparer)	Text.EndsWith("report_2024.csv", ".csv") → true

Extracting Substrings

Substring extraction is essential for parsing structured text like product codes, dates embedded in strings, or address components. M provides four extraction functions:

Text.Start		
Syntax	Text.Start(text as nullable text, count as number) as nullable text	
Parameter	Type	Description
text	nullable text	The source string.
count	number	The number of characters to return from the start.
Returns	Returns the first count characters of text.	
Example	Text.Start("PROD-00142-XL", 4)	
Result	"PROD"	

Text.End		
Syntax	Text.End(text as nullable text, count as number) as nullable text	
Parameter	Type	Description
text	nullable text	The source string.
count	number	The number of characters to return from the end.
Returns	Returns the last count characters of text.	
Example	Text.End("PROD-00142-XL", 2)	
Result	"XL"	

Text.Middle		
Syntax	Text.Middle(text as nullable text, start as number, optional count as nullable number) as nullable text	
Parameter	Type	Description

text	nullable text	The source string.
start	number	Zero-based index of the first character to return.
count	nullable number	Optional. Number of characters to return. If omitted, returns to end of string.

Returns	Returns a substring starting at position start with length count.
Example	Text.Middle("PROD-00142-XL", 5, 5)
Result	"00142"

NOTE Text.Middle is zero-indexed. The first character is at position 0. Text.Middle("hello", 1, 3) returns "ell" — starting at position 1, length 3.

Text.Range

Syntax	Text.Range(text as nullable text, offset as number, optional count as nullable number) as nullable text	
Parameter	Type	Description
text	nullable text	The source string.
offset	number	Zero-based start position.
count	nullable number	Optional. Number of characters. If omitted, returns to end.
Returns	Equivalent to Text.Middle. Returns a range of characters.	
Example	Text.Range("2024-03-15", 0, 4)	
Result	"2024"	
NOTE	Text.Range and Text.Middle are functionally equivalent. Text.Range is often used when the 'range' framing is conceptually clearer (e.g. extracting a year from a date string).	

Searching: Text.PositionOf and Text.PositionOfAny

Text.PositionOf

Syntax	Text.PositionOf(text as text, substring as text, optional occurrence as nullable number, optional comparer as nullable function) as any	
Parameter	Type	Description
text	text	The string to search within.
substring	text	The substring to search for.
occurrence	nullable number	Optional. Occurrence.First (default) or Occurrence.Last.
comparer	nullable function	Optional. Comparer for case control.

Returns	Returns the zero-based index of the first (or last) occurrence of substring in text. Returns -1 if not found.
Example	<code>Text.PositionOf("PROD-00142-XL", "-")</code>
Result	4

NOTE Use `Text.PositionOf` as input to `Text.Middle` or `Text.Start/End` for dynamic extraction: extract everything before the first hyphen with `Text.Start(text, Text.PositionOf(text, "-"))`.

Text.PositionOfAny	Returns the position of the first occurrence of ANY character from a list. Returns -1 if none found.
<code>Text.PositionOfAny(text, characters, optional occurrence)</code>	<code>Text.PositionOfAny("PROD-001/XL", {"-", "/"}) → 4</code>

Modifying Text

Modification functions return a new string with specified characters changed, removed, or replaced. Remember: M strings are immutable — these functions always return a new value, never modifying the original.

Text.Replace		
Syntax	<code>Text.Replace(text as nullable text, old as text, new as text) as nullable text</code>	
Parameter	Type	Description
text	<i>nullable text</i>	The source string.
old	<i>text</i>	The substring to find and replace.
new	<i>text</i>	The replacement string. Use "" to delete.
Returns	Returns a new string with all occurrences of old replaced by new.	
Example	<code>Text.Replace("2024/03/15", "/", "-")</code>	
Result	"2024-03-15"	

NOTE `Text.Replace` replaces ALL occurrences, not just the first. It is case-sensitive. To remove characters, pass "" as the new argument: `Text.Replace(text, " ", "")` removes all spaces.

Text.Remove		
Syntax	<code>Text.Remove(text as nullable text, removeChars as any) as nullable text</code>	
Parameter	Type	Description
text	<i>nullable text</i>	The source string.

<code>removeChars</code>	<i>text or list</i>	A character or list of characters to remove everywhere.
Returns	Returns a new string with all occurrences of every character in <code>removeChars</code> removed.	
Example	<code>Text.Remove("P-R-O-D-001", {"-"})</code>	
Result	"PROD001"	
NOTE <code>Text.Remove</code> removes individual characters (not substrings). To remove a multi-character substring, use <code>Text.Replace</code> with "" as the new value instead.		

Text.Select

Syntax	Text.Select(text as nullable text, selectChars as any) as nullable text	
Parameter	Type	Description
text	nullable text	The source string.
selectChars	text or list	Characters or ranges to keep. All others are removed.
Returns	Returns a string containing only the characters specified in selectChars.	
Example	Text.Select("Tel: +44 (20) 1234-5678", {"0".."9"})	
Result	"442012345678"	
NOTE	Character ranges use the syntax {"0".."9"} for all digits, {"a".."z"} for lowercase letters. Text.Select is the inverse of Text.Remove.	

Text.ReplaceRange

Syntax	<code>Text.ReplaceRange(text as nullable text, offset as number, count as number, newText as text) as nullable text</code>	
Parameter	Type	Description
<code>text</code>	<i>nullable text</i>	The source string.
<code>offset</code>	<i>number</i>	Zero-based position to start replacing.
<code>count</code>	<i>number</i>	Number of characters to replace.
<code>newText</code>	<i>text</i>	The replacement string.
Returns	Replaces <code>count</code> characters starting at <code>offset</code> with <code>newText</code> .	
Example	<code>Text.ReplaceRange("PROD001", 0, 4, "ITEM")</code>	
Result	"ITEM001"	

Text.Insert	Inserts <code>newText</code> into <code>text</code> at the zero-based <code>offset</code> position.
--------------------	---

<code>Text.Insert(text, offset, newText)</code>	<code>Text.Insert("PROD001", 4, "-") → "PROD-001"</code>
Text.Repeat	Returns the text repeated count times. Useful for building separators or fixed-width padding.
<code>Text.Repeat(text, count)</code>	<code>Text.Repeat("-", 5) → "-----"</code>

Splitting Text

Splitting functions convert a text value into a list of text values, dividing at a specified delimiter or set of characters. The result is always a **list** — use list indexing {n} to extract specific parts:

Text.Split

Syntax	Text.Split(text as text, separator as text) as list	
Parameter	Type	Description
text	text	The source string to split.
separator	text	The delimiter string at which to split.
Returns	Returns a list of text values split at every occurrence of separator.	
Example	Text.Split("East,West,Central,South", ",")	
Result	{ "East", "West", "Central", "South" }	
NOTE	Use {0} to get the first part, {1} for the second, etc. Combine with List.Count to check how many parts resulted. Text.Split("hello", ",") with no delimiter in the string returns {"hello"} — a single-element list.	

Practical example: parsing an email address

```
// Common pattern: extract domain from email address
Email = "user@contoso.com",
Parts = Text.Split(Email, "@"), // {"user", "contoso.com"}
Domain = Parts{1}, // "contoso.com"
TLD = Text.Split(Domain, "."){1} // "com"
```

Text.SplitAny	Splits at any character in the separators string. Each character in separators is treated as an individual delimiter.
<code>Text.SplitAny(text, separators)</code>	<code>Text.SplitAny("a,b;c d", ",; ") → {"a", "b", "c", "d"}</code>

Combining Text

Text.Combine

Syntax	<code>Text.Combine(texts as list, optional separator as nullable text) as text</code>
---------------	---

Parameter		Type	Description
texts		list	A list of text values to join.
separator		nullable text	Optional. String inserted between each element. Default: "" (no separator).
Returns	Joins a list of text values into a single string, with an optional separator between each.		
Example	Text.Combine({"John", "Paul", "Jones"}, " ")		
Result	"John Paul Jones"		
NOTE Text.Combine ignores null values in the list — nulls are skipped silently. To include a placeholder for nulls, pre-process with List.Transform: List.Transform(list, each _ ?? "")			

Combining address components:

```
// Build a full address from components
let
    Street = "123 Main St",
    City = "Austin",
    State = "TX",
    Zip = "78701",
    Address = Text.Combine({Street, City, State, Zip}, " ")
// = "123 Main St, Austin, TX, 78701"
in
    Address
```

Converting Values to Text

Text.From

Syntax	Text.From(value as any, optional culture as nullable text) as nullable text	
Parameter	Type	Description
value	any	The value to convert. Works for numbers, dates, logicals, etc.
culture	nullable text	Optional. Culture for formatting. Affects date and number formatting.
Returns	Converts any primitive M value to its text representation.	
Example	Text.From(#date(2024, 3, 15))	
Result	"2024-03-15"	
NOTE	Text.From is the universal value-to-text converter. Use it whenever you need to concatenate a non-text value into a string: "Year: " & Text.From(Date.Year([Date])). Without Text.From, the & operator will error if the right side is not text.	

Text.Format

Syntax	Text.Format(formatString as text, arguments as any, optional culture as nullable text) as text	
Parameter	Type	Description
formatString	text	A format template using #{0}, #{1}, etc. as positional placeholders.
arguments	any	A list of values to substitute, or a single value for #{0}.
culture	text	Optional. Culture for locale-aware number/date formatting.
Returns	Formats a string by substituting values into positional placeholders.	
Example	Text.Format("Order #{0} placed on #{1}", {"CA-001", "2024-03-15"})	
Result	"Order #CA-001 placed on #2024-03-15"	
NOTE	Text.Format is the M equivalent of Python's str.format() or C#'s String.Format(). It is cleaner than multiple & concatenations when building complex strings.	

LESSON 2

Working with Number Functions

After completing this lesson, you will be able to:

- Distinguish between type number, Int64.Type, and Currency.Type in M.
- Apply the four rounding functions and choose the appropriate one for each scenario.
- Use arithmetic functions including Abs, Sign, Mod, and IntegerDivide.
- Apply power, root, exponential, and logarithm functions.
- Use trigonometric functions for geometry and engineering calculations.
- Check special numeric states with Number.IsNaN, Number.IsEven, Number.IsOdd.
- Convert between numbers and text using Number.From and Number.ToText.

Numbers in M: Choosing the Right Numeric Type

Before working with number functions, it is important to understand which numeric type to use. M supports several numeric types, each with different storage characteristics and precision:

Type	Description
type number	64-bit IEEE 754 double-precision floating point. Range: $\sim\pm 1.8 \times 10^{308}$. Precision: ~ 15 -16 significant digits. The default numeric type in M.
Int64.Type	64-bit signed integer. Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. No decimal places. Use for counts, IDs, years, quantities.
Int32.Type	32-bit signed integer. Range: $\sim\pm 2.1$ billion. Smaller memory footprint than Int64 in some contexts.
Currency.Type	128-bit fixed-point decimal with exactly 4 decimal places. Use for monetary amounts to avoid floating-point rounding errors. Maps to SQL money/decimal types.
type decimal	Alias for type number in M. Used interchangeably in type annotations.
Percentage.Type	Stored as a decimal (0.25 = 25%). Displayed as percentage in Power BI visuals.

TIP In Power BI, `Currency.Type` maps to the Fixed decimal number data type, which is processed by the VertiPaq engine more efficiently for financial data. Use `Currency.Type` for any monetary column (Sales, Cost, Profit) to avoid floating-point display artefacts like \$1,249.9999998.

Rounding Functions

M provides four rounding functions, each with distinct behaviour at the midpoint (e.g. 2.5). Choosing the wrong one in a financial context can produce systematic over- or under-reporting:

Number.Round

Syntax	Number.Round(number as nullable number, optional digits as nullable number, optional roundingMode as nullable number) as nullable number	
Parameter	Type	Description
number	<i>nullable number</i>	The value to round.
digits	<i>nullable number</i>	Optional. Decimal places to round to. Default: 0 (round to integer).
roundingMode	<i>nullable number</i>	Optional. RoundingMode.AwayFromZero (default) or RoundingMode.ToEven (banker's rounding).
Returns	Rounds number to the specified number of decimal places.	
Example	Number.Round(2.5675, 2)	
Result	2.57	
NOTE	RoundingMode.ToEven (banker's rounding) rounds 2.5 to 2 and 3.5 to 4 — always rounding to the nearest even number at the midpoint. This reduces systematic rounding bias in large financial datasets. Use it when statistical accuracy matters.	

Number.RoundUp

Syntax	Number.RoundUp(number as nullable number, optional digits as nullable number) as nullable number	
Parameter	Type	Description
number	<i>nullable number</i>	The value to round.
digits	<i>nullable number</i>	Optional. Decimal places. Default: 0.
Returns	Always rounds away from zero (towards positive infinity for positive numbers).	
Example	Number.RoundUp(2.001, 2)	
Result	2.01	

NOTE `Number.RoundUp(2.5, 0) = 3`, `Number.RoundUp(-2.5, 0) = -2`. Use for calculations that must never undercount (e.g. packaging units, ceiling pricing).

Number.RoundDown

Syntax `Number.RoundDown(number as nullable number, optional digits as nullable number) as nullable number`

Parameter	Type	Description
<code>number</code>	<i>nullable number</i>	The value to round.
<code>digits</code>	<i>nullable number</i>	Optional. Decimal places. Default: 0.
Returns	Always rounds toward zero (towards negative infinity for positive numbers — floor).	
Example	<code>Number.RoundDown(2.999, 0)</code>	
Result	2	

NOTE `Number.RoundDown(2.5, 0) = 2`. Use for floor calculations: age from birthdate, full years of service, complete units shipped.

Number.RoundTowardZero

Syntax `Number.RoundTowardZero(number as nullable number, optional digits as nullable number) as nullable number`

Parameter	Type	Description
<code>number</code>	<i>nullable number</i>	The value to round.
<code>digits</code>	<i>nullable number</i>	Optional. Decimal places. Default: 0.
Returns	Truncates toward zero (removes the decimal portion). Equivalent to <code>Number.RoundDown</code> for positive numbers.	
Example	<code>Number.RoundTowardZero(-2.9)</code>	
Result	-2	

NOTE For negative numbers: `Number.RoundDown(-2.9) = -3`, but `Number.RoundTowardZero(-2.9) = -2`. This distinction matters when working with negative financial values.

Arithmetic Functions

Number.Abs

Syntax `Number.Abs(number as nullable number) as nullable number`

Parameter	Type	Description
number	<i>nullable number</i>	The input value.
Returns	Returns the absolute (non-negative) value.	
Example	Number.Abs(-142.5)	
Result	142.5	
NOTE Use in financial calculations to compute absolute variances: Number.Abs([Budget] - [Actual]) gives the magnitude of the difference regardless of direction.		

Number.Sign

Syntax	Number.Sign(number as nullable number) as nullable number	
Parameter	Type	Description
number	nullable number	The input value.
Returns	Returns 1 if positive, -1 if negative, 0 if zero.	
Example	Number.Sign(-42)	
Result	-1	
NOTE	Use Number.Sign to determine direction without caring about magnitude. Useful for classifying rows as favourable/unfavourable variances.	

Number.Mod

Syntax	Number.Mod(number as nullable number, divisor as number) as nullable number	
Parameter	Type	Description
number	nullable number	The dividend.
divisor	number	The divisor. Must not be zero.
Returns	Returns the remainder after dividing number by divisor.	
Example	Number.Mod(17, 5)	
Result	2	
NOTE	Common uses: Number.Mod([RowIndex], 2) = 0 identifies even rows. Number.Mod(Date.Month([Date]) - 1, 3) + 1 calculates the month-within-quarter (1, 2, or 3).	

Number.IntegerDivide

Syntax	Number.IntegerDivide(number as nullable number, divisor as number) as nullable number	
Parameter	Type	Description
number	nullable number	The dividend.
divisor	number	The divisor.
Returns	Returns the integer quotient (floor division). Discards the remainder.	
Example	Number.IntegerDivide(17, 5)	
Result	3	
NOTE	Combine with Number.Mod: Number.IntegerDivide(minutes, 60) gives full hours; Number.Mod(minutes, 60) gives the remaining minutes.	

Power, Root, and Logarithm Functions

Number.Power

Syntax	Number.Power(number as nullable number, power as nullable number) as nullable number	
Parameter	Type	Description
number	nullable number	The base.
power	nullable number	The exponent.
Returns	Returns number raised to the specified power.	
Example	Number.Power(2, 10)	
Result	1024	
NOTE	Use Number.Power(value, 1/3) for cube roots, Number.Power(value, 1/n) for nth roots. Compound interest formula: Principal * Number.Power(1 + Rate, Years).	

Number.Sqrt

Syntax	Number.Sqrt(number as nullable number) as nullable number	
Parameter	Type	Description
number	<i>nullable number</i>	The value. Must be non-negative for a real result.
Returns	Returns the square root. Returns NaN for negative inputs.	
Example	Number.Sqrt(144)	
Result	12	

NOTE Equivalent to `Number.Power(number, 0.5)`. Returns `Number.NaN` for negative inputs without raising an error — check with `Number.IsNaN` if the input might be negative.

Number.Exp	Returns e (Euler's number, ~2.71828) raised to the given power.
<code>Number.Exp(number)</code>	<code>Number.Exp(1)</code> → 2.718... (e)
Number.Log	Returns the logarithm of number to the specified base. Default base is e (natural log).
<code>Number.Log(number, optional base)</code>	<code>Number.Log(8, 2)</code> → 3
Number.Log10	Returns the base-10 (common) logarithm. Equivalent to <code>Number.Log(number, 10)</code> .
<code>Number.Log10(number)</code>	<code>Number.Log10(1000)</code> → 3

Trigonometric Functions

Trigonometric functions operate in **radians**, not degrees. To convert degrees to radians: radians = degrees * `Number.PI` / 180. These are used in geospatial calculations, engineering models, and signal processing:

Number.Sin	Sine of the angle in radians.
<code>Number.Sin(number)</code>	<code>Number.Sin(Number.PI / 2)</code> → 1
Number.Cos	Cosine of the angle in radians.
<code>Number.Cos(number)</code>	<code>Number.Cos(0)</code> → 1
Number.Tan	Tangent of the angle in radians.
<code>Number.Tan(number)</code>	<code>Number.Tan(Number.PI / 4)</code> → 1
Number.Asin	Arcsine (inverse sine). Result is in radians.
<code>Number.Asin(number)</code>	<code>Number.Asin(1)</code> → 1.5707... (pi/2)
Number.Acos	Arccosine (inverse cosine). Result is in radians.
<code>Number.Acos(number)</code>	<code>Number.Acos(1)</code> → 0
Number.Atan	Arctangent (inverse tangent). Result is in radians.
<code>Number.Atan(number)</code>	<code>Number.Atan(1)</code> → 0.7853... (pi/4)
Number.Atan2	Two-argument arctangent. Handles all quadrants. Used in bearing calculations.
<code>Number.Atan2(y, x)</code>	<code>Number.Atan2(1, 1)</code> → 0.7853...

Numeric Constants and Special Values

Constant	Value	Notes
<code>Number.PI</code>	3.14159265358979...	The mathematical constant pi.
<code>Number.E</code>	2.71828182845904...	Euler's number, the base of the natural logarithm.
<code>Number.Epsilon</code>	4.94065645841247E-324	The smallest positive number representable as a double.
<code>Number.NaN</code>	NaN (Not a Number)	The IEEE 754 Not-a-Number value. Result of invalid operations like 0/0 or Sqrt(-1).
<code>Number.PositiveInfinity</code>	Infinity	Positive infinity. Result of operations like 1/0.
<code>Number.NegativeInfinity</code>	-Infinity	Negative infinity. Result of operations like -1/0.

Checking Numeric States

<code>Number.IsNaN</code>	Returns true if the value is NaN. Use this before any calculation that might produce NaN.
<code>Number.IsNaN(number)</code>	<code>Number.IsNaN(0/0) → true</code>
<code>Number.IsEven</code>	Returns true if the integer is even. Useful for alternating row logic or biweekly period checks.
<code>Number.IsEven(number)</code>	<code>Number.IsEven(4) → true</code>
<code>Number.IsOdd</code>	Returns true if the integer is odd. Complement to <code>Number.IsEven</code> .
<code>Number.IsOdd(number)</code>	<code>Number.IsOdd(7) → true</code>

Number Conversion

Number.From		
Syntax	<code>Number.From(value as any, optional culture as nullable text) as nullable number</code>	
Parameter	Type	Description
value	any	The value to convert. Works for text, logical, date, datetime.
culture	nullable text	Optional culture for parsing locale-specific number formats.
Returns	Converts a value to type number. Returns null if conversion is not possible.	
Example	<code>Number.From("1,234.56", "en-US")</code>	
Result	1234.56	

NOTE Use `Number.From` inside `try...otherwise` to safely convert text that might not be numeric: `try Number.From([Column]) otherwise null`.

Number.ToText	Converts a number to text with a specified format string. Supports standard .NET format specifiers.
<code>Number.ToText(number, optional format, optional culture)</code>	<code>Number.ToText(1234567.89, "#,##0.00")</code> → "1,234,567.89"

LESSON 3

Conditional Logic in M

After completing this lesson, you will be able to:

- Write correct if-then-else expressions in M.
- Nest conditional expressions to handle multiple branches.
- Combine conditions using and, or, and not operators.
- Write null-safe conditional expressions.
- Apply conditional logic inside Table.AddColumn to create classification columns.
- Explain short-circuit evaluation in M conditional expressions.

The if-then-else Expression

M's conditional expression uses the keywords **if**, **then**, and **else**. Unlike Excel's IF() function, M's conditional is an expression — it always returns a value, and both the then and else branches must return compatible types:

if-then-else syntax:

```
// Basic syntax
if condition then trueValue else falseValue

// Simple example
Status = if [Sales] > 1000 then "High" else "Normal"

// With numbers
AbsValue = if [Profit] >= 0 then [Profit] else -[Profit]

// The else branch is MANDATORY in M
// This is INVALID – will produce a parse error:
// if [Sales] > 1000 then "High" <-- missing else
```

WARNING The **else** branch is not optional in M. Every if expression must have an else. If you genuinely want 'do nothing' when the condition is false, return null: if condition then value else null

Nested Conditional Expressions

Nested if-then-else expressions handle multiple branches. Because M is expression-based, nesting happens in the else branch. Proper indentation is critical for readability:

Nested if-then-else for multi-tier classification:

```
// Three-tier sales classification
SalesTier =
if [Sales] >= 10000 then "Platinum"
else if [Sales] >= 5000 then "Gold"
else if [Sales] >= 1000 then "Silver"
else "Bronze"
```

Using nested if inside Table.AddColumn:

```
// Full Table.AddColumn context
AddTier = Table.AddColumn(Source, "SalesTier",
each
if [Sales] >= 10000 then "Platinum"
else if [Sales] >= 5000 then "Gold"
else if [Sales] >= 1000 then "Silver"
else "Bronze",
type text)
```

TIP M evaluates branches **top to bottom**. As soon as a condition is true, that branch is returned and evaluation stops. Order your conditions from most specific (highest threshold) to most general (lowest threshold).

Combining Conditions: and, or, not

M uses the lowercase keywords **and**, **or**, and **not** for logical operators (not &&, ||, ! as in other languages):

Logical operators in M:

```
// AND – both conditions must be true
WestHighSales = [Region] = "West" and [Sales] > 5000

// OR – at least one condition must be true
IsCoastal = [Region] = "West" or [Region] = "East"

// NOT – inverts a logical value
IsNotReturned = not [IsReturned]

// Combining and/or – use parentheses to control precedence
IsPremium = ([Category] = "Technology" or [Category] = "Furniture")
and [Sales] > 1000

// In Table.SelectRows
FilteredRows = Table.SelectRows(Source,
each ([Region] = "West" or [Region] = "East")
and [Sales] > 1000
and not [IsReturned])
```

WARNING Operator precedence in M: **not** binds tightest, then **and**, then **or**. Always use parentheses when mixing and and or to make intent explicit and avoid subtle bugs: A or B and C means A or (B and C), which may not be what you intend.

Null-Safe Conditionals

One of the most common sources of unexpected results in M is forgetting to handle null values in conditions. In M, null comparisons behave differently from some other languages:

Null handling in conditional expressions:

```
// null comparisons always return null (not true or false)
null > 100 // = null (NOT false)
null = null // = null (NOT true!)
null <> null // = null (NOT false)

// This means a filter can silently drop null rows:
Table.SelectRows(t, each [Sales] > 100)
// Rows where [Sales] is null are EXCLUDED (null > 100 = null, treated as false)

// To include nulls, be explicit:
Table.SelectRows(t, each [Sales] = null or [Sales] > 100)

// The ?? operator: null coalescing
// Returns the left side if not null, otherwise the right side
SafeSales = [Sales] ?? 0 // If Sales is null, use 0

// Null-safe conditional pattern:
Tier = if [Sales] = null then "Unknown"
else if [Sales] >= 1000 then "High"
else "Low"
```

TIP The ?? operator (null-coalescing) is one of M's most useful features for defensive coding. Use it to replace nulls with defaults: [Discount] ?? 0, [Country] ?? "Unknown". It is equivalent to: if [Column] = null then default else [Column].

Short-Circuit Evaluation in Conditions

M evaluates the branches of if-then-else lazily: only the branch that is selected is evaluated. The other branch is not evaluated at all. This enables safe patterns like dividing only when the denominator is non-zero:

Short-circuit evaluation:

```
// Safe division – the division only runs when Quantity > 0
UnitCost = if [Quantity] = 0 then null
else [TotalCost] / [Quantity]

// Without short-circuit, you would get a division-by-zero error
// But because else only evaluates when Quantity <> 0, this is safe
```

```
// Important: 'and' and 'or' do NOT short-circuit in M
// Both sides are evaluated:
// [A] = 0 and Number.Sqrt([B]) > 0 -- Sqrt runs even if A = 0
// Use nested if for true short-circuit with and/or
```

LESSON 4

The each Keyword and Row-Level Operations

After completing this lesson, you will be able to:

- Explain what each means as M syntax sugar.
- Use the underscore `_` as the implicit context variable.
- Apply each correctly in `Table.AddColumn` vs `Table.SelectRows`.
- Write nested each expressions for list and record operations.
- Identify situations where each should NOT be used.

What each Really Means

The **each** keyword is one of the most frequently misunderstood features of M. It is purely **syntactic sugar** — a shorthand notation that the M parser expands into a full function expression before evaluation. Understanding this expansion completely removes the mystery:

each expansion:

```
// 'each' is shorthand for a single-parameter function

// These two are IDENTICAL:
each [Sales] > 1000
(_) => _[Sales] > 1000

// And these two are IDENTICAL:
each Text.Upper([Name])
(_) => Text.Upper(_[Name])

// The underscore _ is the implicit parameter name
// When used inside a table context, _ represents the current ROW (a record)
// When used inside a list context, _ represents the current ELEMENT
```

The Underscore `_` as Context Variable

The underscore `_` represents the context variable passed to each. Its meaning depends on where each is used:

Table.AddColumn	<code>_</code> is the current row as a record. <code>[ColumnName]</code> is shorthand for <code>_[ColumnName]</code> .
------------------------	--

Example: `Table.AddColumn(t, "Tax", each [Sales] * 0.1) -- _ is a row record; [Sales] = _[Sales]`

Table.SelectRows `_` is the current row as a record. [ColumnName] accesses row fields.

Example: `Table.SelectRows(t, each [Region] = "West") -- keeps rows where the Region field equals "West"`

List.Select `_` is the current list element (not a record).

Example: `List.Select({1,2,3,4,5}, each _ > 3) -- returns {4, 5}; _ is each number`

List.Transform `_` is the current list element.

Example: `List.Transform({1,2,3}, each _ * 2) -- returns {2,4,6}`

Table.TransformColumns `_` is the current cell value in the specified column.

Example: `Table.TransformColumns(t, {"Name", each Text.Upper(_)}) -- _ is each text value`

When NOT to Use each

The `each` shorthand works only for single-parameter anonymous functions. There are three situations where you must use the full `(_) => expression` syntax instead:

- **Multiple parameters:** Functions that take more than one argument cannot use `each`.
Example: `Table.AddColumn(t, "Combined", (row) => row[First] & " " & row[Last])`. Here, using `each` would work too, but `(row) =>` makes intent clearer.
- **Name collision:** If an inner `each` is nested inside an outer `each`, the inner `_` would shadow the outer `_`. Use explicit parameter names: `(outerRow) => List.Select(someList, each _ > outerRow[Threshold])`.
- **Recursive functions:** Self-referential functions cannot use `each` — they need named parameters and the `@` recursive reference operator (covered in Module 10).

Avoiding each shadowing in nested contexts:

```
// PROBLEM: inner each shadows outer each – outer _ is inaccessible
Table.AddColumn(Source, "HasMatch",
each List.Contains(ReferenceList, each _ = [ID])) // WRONG - inner each

// SOLUTION: use explicit parameter name for the outer function
Table.AddColumn(Source, "HasMatch",
(row) => List.Contains(ReferenceList, each _ = row[ID])) // CORRECT
```

LESSON 5

Combining Text, Numbers, and Logic in Real Queries

After completing this lesson, you will be able to:

- Use `Table.TransformColumns` to apply text and number transformations in bulk.
- Build a complete product name standardisation pipeline.
- Build a revenue classification pipeline using nested conditionals.
- Parse structured data from free-text fields using text extraction functions.
- Chain multiple `Table.AddColumn` steps to build derived analytical columns.

Table.TransformColumns — Bulk Transformation

When you need to apply the same transformation to multiple columns, or a different transformation to each column, **Table.TransformColumns** is more efficient than multiple `Table.AddColumn` + `Table.RemoveColumns` pairs. It transforms each specified column **in place**, replacing the original values:

Table.TransformColumns with multiple column transformations:

```
// Apply different transformations to multiple columns at once
Cleaned = Table.TransformColumns(Source, {
// {column name, transformation function, optional output type}
{"ProductName", each Text.Proper(Text.Trim(_)), type text},
{"CustomerName", each Text.Proper(Text.Trim(_)), type text},
{"Region", each Text.Upper(Text.Trim(_)), type text},
{"Sales", each Number.Round(_, 2), type number},
{"Discount", each Number.Round(_ * 100, 1), type number} // to %
})
```

TIP `Table.TransformColumns` transforms columns that already exist. `Table.AddColumn` creates new columns while keeping the originals. Use `TransformColumns` when replacing; use `AddColumn` when you need both the original and the transformed version.

Real-World Pattern: Product Name Standardisation

Product names arriving from ERP systems are notoriously inconsistent: extra spaces, all-caps, dashes instead of spaces, product code prefixes mixed with names. Here is a complete standardisation pipeline:

Complete product name standardisation pipeline:

```

let
Source = /* Load product data */,

// Step 1: Basic cleanup – trim whitespace and fix casing
BasicClean = Table.TransformColumns(Source, {
{"ProductName", each Text.Proper(Text.Trim(_)), type text}
}),

// Step 2: Remove product code prefix (format: 'PROD-XXXXX - Description')
// Extract everything after ' - '
RemoveCode = Table.TransformColumns(BasicClean, {
{"ProductName", each
let
SepPos = Text.PositionOf(_, " - "),
// If separator found, take everything after it
// If not found (no prefix), keep the whole name
Result = if SepPos = -1 then _
else Text.Middle(_, SepPos + 3)
in
Text.Trim(Result),
type text}
}),

// Step 3: Replace internal dashes with spaces (e.g. 'Stapler-Black' -> 'Stapler
Black')
FixDashes = Table.TransformColumns(RemoveCode, {
{"ProductName", each Text.Replace(_, "-", " "), type text}
}),

// Step 4: Collapse double spaces (some names have ' ' after replacements)
CollapseSpaces = Table.TransformColumns(FixDashes, {
{"ProductName",
each Text.Combine(
List.Select(
Text.Split(_, " "),
each _ <> "" // Remove empty strings from double-space splits
),
" " // Rejoin with single space
),
type text}
})
in
CollapseSpaces

```

Real-World Pattern: Revenue Classification Pipeline

A typical business requirement: derive multiple analytical columns from a revenue figure — absolute margin, tier classification, flag for target attainment, and a formatted display label:

Full revenue classification pipeline:

```

let
Source = /* Fact table with Sales, Cost, Target columns */,

// Step 1: Calculate gross profit and margin %
AddProfit = Table.AddColumn(Source, "GrossProfit",
each [Sales] - [Cost], type number),

AddMargin = Table.AddColumn(AddProfit, "MarginPct",
each if [Sales] = 0 then null // null-safe: avoid divide by zero
else Number.Round(([Sales] - [Cost]) / [Sales] * 100, 1),
type number),

// Step 2: Classify into performance tiers
AddTier = Table.AddColumn(AddMargin, "PerfTier",
each if [MarginPct] = null then "No Data"
else if [MarginPct] >= 30 then "Platinum"
else if [MarginPct] >= 20 then "Gold"
else if [MarginPct] >= 10 then "Silver"
else if [MarginPct] >= 0 then "Bronze"
else "Loss",
type text),

// Step 3: Flag whether target was hit (null-safe)
AddTargetFlag = Table.AddColumn(AddTier, "TargetMet",
each ([Sales] ?? 0) >= ([Target] ?? 0),
type logical),

// Step 4: Build a display label combining text and numbers
AddLabel = Table.AddColumn(AddTargetFlag, "SummaryLabel",
each Text.Format(
"#{0} | #{1}% | #{2}",
{[PerfTier],
Text.From([MarginPct] ?? 0),
if [TargetMet] then "On Target" else "Below Target"}
),
type text)
in
AddLabel

```

Real-World Pattern: Address Parsing

Free-text address fields often need to be parsed into components. This pattern uses `Text.Split`, `Text.PositionOf`, and conditional logic together:

Address parsing with Text functions and conditionals:

```

// Input: "123 Main St, Austin, TX 78701"
// Goal: Street, City, State, ZipCode as separate columns

AddStreet = Table.AddColumn(Source, "Street",
each Text.Trim(Text.Split([Address], ",") {0}),

```

```
type text),

AddCity = Table.AddColumn(AddStreet, "City",
each if Text.Contains([Address], ",")
then Text.Trim(Text.Split([Address], ","){1})
else null,
type text),

AddStateZip = Table.AddColumn(AddCity, "StateZip",
each if List.Count(Text.Split([Address], ",")) >= 3
then Text.Trim(Text.Split([Address], ","){2})
else null,
type text),

AddState = Table.AddColumn(AddStateZip, "State",
each if [StateZip] <> null
then Text.Start(Text.Trim([StateZip]),
Text.PositionOf(Text.Trim([StateZip]), " "))
else null,
type text),

AddZip = Table.AddColumn(AddState, "ZipCode",
each if [StateZip] <> null
then Text.End(Text.Trim([StateZip]), 5)
else null,
type text)
```

DEMONSTRATION 1**Building a Name and Address Cleanup Pipeline**

In this demonstration, the instructor builds a complete data cleaning pipeline for customer name and address data, using `Text.Trim`, `Text.Proper`, `Text.Split`, `Text.PositionOf`, and `Text.Combine` in a realistic business scenario.

Scenario: The Contoso CRM system exports customer data with inconsistent name casing, trailing whitespace, and a single `FullName` field that must be split into `FirstName` and `LastName`.

Demonstration Steps**Step 1: Load the Contoso_Products.csv file**

Open Power BI Desktop → Get Data → Text/CSV. Navigate to `Contoso_Products.csv`. Click Transform Data. Observe the raw data — note the inconsistent casing and extra spaces.

Step 2: Open the Advanced Editor

Click Home → Advanced Editor. Examine the auto-generated M. Add descriptive step names.

Step 3: Add a bulk text cleanup step

After `PromotedHeaders`, add a `TransformColumns` step that applies `Text.Proper(Text.Trim(_))` to the `CustomerName` and `ProductName` columns simultaneously.

Step 4: Split FullName into components

Add a `Table.AddColumn` step for `FirstName` using `Text.Start` and `Text.PositionOf` to extract the text before the first space. Add a second step for `LastName` using `Text.Middle` to get everything after the first space.

Step 5: Handle edge cases — single-word names

Wrap the extraction in an if expression: `if Text.Contains([FullName], " ") then... else [FullName]`. This prevents errors when no space exists.

Step 6: Rebuild a CleanFullName from the parts

Add a final step using `Text.Combine({[FirstName], [LastName]}, " ")` to verify the split and rejoin works.

Step 7: Inspect the Formula Bar after each step

Click through each Applied Step and observe how the M expression changes in the Formula Bar. Note how complex nested functions appear.

Step 8: Click Done and review the result

Close the Advanced Editor. Verify: `CustomerName` is properly cased, `FirstName` and `LastName` are correctly split, `CleanFullName` matches the original (cleaned) `FullName`.

RESULT The query produces a clean customer table with: properly cased `CustomerName`, separated `FirstName` and `LastName` columns, and a `CleanFullName` verification column. All Applied Steps are named descriptively.

DEMONSTRATION 2**Creating a Multi-Tier Conditional Classification Column**

In this demonstration, the instructor builds a four-tier revenue classification system using nested if-then-else, null-safe conditions, and the Text.Format function to produce a formatted summary column.

Scenario: The Superstore FactSales table needs a MarginTier column (Platinum/Gold/Silver/Bronze/Loss/No Data) and a display label for use in a Power BI tooltip.

Demonstration Steps**Step 1: Open the FactSales query**

In Power Query Editor, select the FactSales query from the Queries pane.

Step 2: Calculate GrossMarginPct in a new column

Add: Table.AddColumn(ProfitableOrders, "MarginPct", each if [Sales] = 0 then null else Number.Round([Profit] / [Sales] * 100, 1), type number). Observe that null-safety is built in for the Sales = 0 case.

Step 3: Add the MarginTier classification

Add a nested if-then-else with 6 branches: null → No Data, >=30 → Platinum, >=20 → Gold, >=10 → Silver, >=0 → Bronze, else → Loss.

Step 4: Test with boundary values

Temporarily change the in clause to preview only rows where MarginPct is 0, null, and the tier boundaries. Verify each boundary maps to the correct tier.

Step 5: Add a formatted SummaryLabel column

Use Text.Format to build: [Tier] | [MarginPct]% | [TargetStatus]. Demonstrate null coalescing with ?? for the MarginPct display.

Step 6: Review performance implications

Discuss: these derived columns run in M at refresh time. For interactive slicers on tier, a DAX calculated column would be more appropriate. M columns are best for classification that doesn't depend on filter context.

RESULT FactSales now has MarginPct (number), MarginTier (text), and SummaryLabel (text) columns. All boundary cases including null and zero Sales handle gracefully without errors.

LAB 2

Text, Numbers, and Conditional Logic

In this lab you will apply all text functions, number functions, and conditional logic from Module 2 to build a complete data cleaning and enrichment pipeline. You will work with both the Superstore dataset and the Contoso Products file.

Lab Scenario

You are a Power BI developer at Contoso Analytics. Your manager has assigned you two tasks: (1) clean and standardise the product master data arriving from the legacy ERP system, and (2) enrich the sales fact table with margin calculations, tier classifications, and customer-facing display labels for the executive dashboard.

NOTE Continue from the FactSales query you built in Lab 1. You will extend it with additional columns. Also ensure Contoso_Products.csv is loaded as a separate query named ContosoProducts before starting Exercise 1.

Exercise 1: Text Transformations on Product and Customer Data (Estimated time: 20 minutes)

In this exercise you will clean the ContosoProducts query using text functions.

1. Select the **ContosoProducts** query in the Queries pane.
2. Open the Advanced Editor. After PromotedHeaders, add a TransformColumns step that applies **Text.Trim** to every text column (ProductName, Category, SubCategory, Supplier).
3. Add a second TransformColumns step that applies **Text.Proper** to ProductName and Supplier, and **Text.Upper** to Category.
4. Add a new column **ProductCode** that extracts the first 8 characters of ProductName using **Text.Start**. If ProductName is shorter than 8 characters, return the full name.
5. Add a column **HasDiscount** (type logical) that is true if the Supplier name **contains** the word 'discount' (case-insensitive). Use Text.Contains with Comparer.OrdinalIgnoreCase.
6. Add a column **SKU** that zero-pads the ProductID to 8 digits using **Text.PadStart** with '0'. ProductID is currently a number — use Text.From first.
7. Add a column **DisplayName** using **Text.Format**: format it as "[Category] | [ProductName] (SKU: [SKU])". Use Text.Format with #{0}, #{1}, #{2} placeholders.
8. Verify: no ProductName has leading or trailing spaces. Use Column Quality view to confirm 100% valid in all text columns.
9. Rename the query to **DimProduct**. Ensure Enable load is checked.

RESULT DimProduct has clean, consistently cased text columns. ProductCode shows the first 8 characters of each product name. SKU is a zero-padded 8-digit string. DisplayName shows a formatted label like: 'TECHNOLOGY | Laptop Pro 15 (SKU: 00000042)'. HasDiscount is true for supplier names containing 'discount'.

Exercise 2: Number Calculations — Margins, Rounding, and Derivations (Estimated time: 20 minutes)

In this exercise you will add calculated numeric columns to the FactSales query.

1. Select the **FactSales** query.
2. After the ProfitableOrders step, add a column **GrossProfit** = [Sales] - [Cost] (if Cost exists) or [Profit] (if Profit exists in your dataset). Type: type number.
3. Add a column **MarginPct**: divide GrossProfit by Sales * 100, rounded to 1 decimal place using **Number.Round**. Handle the case where Sales = 0 by returning null.
4. Add a column **MarginPctDisplay**: the same value rounded to 0 decimal places using **Number.Round**. This will be used for display in a visual.
5. Add a column **SalesRoundedUp** using **Number.RoundUp** to the nearest 100. (Hint: `Number.RoundUp([Sales] / 100, 0) * 100`.)
6. Add a column **DiscountAmount**: if a Discount column exists, calculate Sales * Discount. If Discount is null, treat as 0 using the ?? operator.
7. Add a column **AbsVariance**: the absolute difference between Sales and a fixed target of 5000 using **Number.Abs**.
8. Add a column **IsEvenRow**: add an index column first using **Table.AddIndexColumn**, then use **Number.IsEven** on the index to create a logical column for alternating row shading.
9. Verify all new columns have the correct types (number or logical) and no unexpected nulls.

RESULT FactSales has MarginPct (rounded to 1dp), MarginPctDisplay (rounded to 0dp), SalesRoundedUp (nearest 100), DiscountAmount (null-safe), AbsVariance, and IsEvenRow. No division-by-zero errors when Sales = 0.

Exercise 3: Conditional Logic — Classify, Flag, and Tier (Estimated time: 20 minutes)

In this exercise you will add classification and flag columns using if-then-else logic.

1. Continuing in **FactSales**, add a column **SalesTier** using a **nested if-then-else** with four tiers: Platinum (Sales >= 10000), Gold (>= 5000), Silver (>= 1000), Bronze (< 1000). Type: type text.
2. Add a column **MarginTier** using a nested if based on MarginPct: null → 'No Data', >= 30 → 'High Margin', >= 15 → 'Mid Margin', >= 0 → 'Low Margin', else → 'Loss'. Type: type text.
3. Add a column **IsHighValue** (logical): true if Sales >= 5000 **and** MarginPct >= 20. Handle nulls — use MarginPct ?? 0 to avoid null comparisons.

4. Add a column **RegionGroup**: 'Coastal' if Region is 'East' or 'West', 'Central' if Region is 'Central', 'South' if Region is 'South', else 'Other'. Use **or** for the first branch.
5. Add a column **ShippingFlag**: 'Express' if Quantity >= 10, 'Standard' if Quantity >= 5, 'Basic' otherwise. (Quantity must be non-null — use ?? 0.)
6. Add a column **PerformanceLabel** using **Text.Format** to produce a label: '[SalesTier] – [MarginTier]'. Use **Text.Format** with #{0} and #{1}.
7. Test boundary values: temporarily filter to rows where Sales = 10000, Sales = 4999, Sales = 1000, Sales = 999 and verify SalesTier is correct at each boundary.
8. Remove the test filter and restore the ProfitableOrders filter.

RESULT FactSales has SalesTier, MarginTier, IsHighValue, RegionGroup, ShippingFlag, and PerformanceLabel columns. All boundary values are classified correctly. No null-related errors in any conditional column.

Exercise 4: Full Pipeline — Clean, Compute, Classify, and Load (Estimated time: 25 minutes)

In this exercise you will review your complete pipeline, optimise the step order, and load the final model into Power BI Desktop.

1. Review the Applied Steps pane for both FactSales and DimProduct. Ensure all steps have descriptive names (no #"Changed Type" style names).
2. In FactSales, verify that **Table.SelectColumns** (keeping only needed columns) comes **before** any AddColumn steps. This ensures you are not carrying unnecessary columns through the pipeline.
3. Add a final step to FactSales that reorders columns logically: key columns first (Order ID, Order Date, Customer Name, Region), then measures (Sales, Profit, Quantity, Discount), then derived (MarginPct, SalesTier, MarginTier, etc.). Use **Table.SelectColumns** with the full ordered list.
4. Right-click the original auto-generated steps (Source, Promoted Headers) in Applied Steps and rename them to Source and PromoteHeaders.
5. Open the Advanced Editor for FactSales. Add a comment block at the top of the let block documenting: query purpose, author, last modified date, and source.
6. Verify DimProduct has Enable load = true and FactSales has Enable load = true. All intermediate/staging queries should have Enable load = false.
7. Click **Close & Apply**. Allow all queries to load.
8. In Power BI Desktop, switch to **Data View**. Verify FactSales shows all calculated columns (MarginPct, SalesTier, MarginTier, PerformanceLabel) with correct values.
9. Switch to **Model View**. Create a relationship between DimProduct[ProductName] and FactSales[ProductName] (or on a common key if available).

10. Create a quick table visual in Report View with SalesTier and a count of Order IDs. Verify all four tiers appear and the counts make sense.

RESULT Both FactSales and DimProduct are loaded into the Power BI data model. FactSales has a full set of derived columns: MarginPct, SalesTier, MarginTier, IsHighValue, RegionGroup, ShippingFlag, and PerformanceLabel. A table visual confirms all four SalesTier values are present. The Applied Steps pane shows clean, descriptively named steps for both queries.

Module Review and Takeaways

Review Questions

What is the difference between `Text.Replace` and `Text.Remove`?

Answer: `Text.Replace` replaces all occurrences of a substring with another string.

`Text.Remove` removes individual characters from a list — it removes every occurrence of each specified character, but cannot remove multi-character substrings.

How would you extract only the digits from a mixed text-number field like 'Order-1234-B'?

Answer: Use `Text.Select` with a character range: `Text.Select([Field], {"0".. "9"})` returns '1234'.

Alternatively, use `Text.Remove` with all non-digit characters.

What is the difference between `Number.Round`, `Number.RoundUp`, and `Number.RoundDown` at the value 2.5?

Answer: `Number.Round(2.5, 0) = 3` (rounds away from zero by default). `Number.RoundUp(2.5, 0) = 3` (always rounds up / away from zero). `Number.RoundDown(2.5, 0) = 2` (always rounds down / toward zero). `Number.Round` with `RoundingMode.ToEven` returns 2 (banker's rounding — rounds to nearest even).

What does the `??` operator do in M?

Answer: The null-coalescing operator `??` returns the left operand if it is not null, otherwise returns the right operand. `[Sales] ?? 0` means: use `[Sales]` if not null, otherwise 0.

What does 'each `[Sales] > 1000`' expand to as a full M expression?

Answer: `(_) => _[Sales] > 1000` — a single-parameter anonymous function where `_` is the current row record, and `[Sales]` is shorthand for `_[Sales]`.

When should you use `Table.TransformColumns` instead of `Table.AddColumn`?

Answer: Use `Table.TransformColumns` when replacing existing column values in-place. Use `Table.AddColumn` when you need to keep the original column and add a new derived one.

What is the risk of writing 'if `[Sales] > 0` and `[Sales] / [Cost] > 2` then...' in M?

Answer: The 'and' operator in M does not short-circuit. Both sides are evaluated even if the left side is false. If `[Cost] = 0`, the division `[Sales]/[Cost]` throws an error even when `[Sales] = 0` would make the and false. Use a nested if instead: `if [Sales] > 0 then if [Sales] / [Cost] > 2 then... else... else...`

How do you make a `Text.Contains` search case-insensitive?

Answer: Pass `Comparer.OrdinalIgnoreCase` as the third argument: `Text.Contains([Field], "search", Comparer.OrdinalIgnoreCase)`. Without this, the comparison is case-sensitive by default.

Best Practices

- Always chain `Text.Trim` with `Text.Proper` or `Text.Upper` — trim first, then case-convert. Reversing the order can leave whitespace in place.

- Use `Text.Contains` with `Comparer.OrdinalIgnoreCase` for all user-facing text searches. Case-sensitive comparisons are a common source of missed matches.
- Prefer `Currency.Type` over `type number` for monetary columns. It prevents floating-point display artefacts and is processed more efficiently by VertiPaq.
- Always handle null before performing arithmetic. Use the `??` operator or an explicit null check in an if expression before dividing or comparing.
- In nested if-then-else, put the most specific condition first (highest threshold or most restrictive). M evaluates top-to-bottom and returns on the first true branch.
- Use `Number.Round(value, 2)` for financial figures displayed to users. Use `RoundingMode.ToEven` in statistical/scientific contexts to avoid systematic rounding bias.
- Never use 'and' for short-circuit guarding in M. The and operator evaluates both sides. Use nested if expressions when the second condition depends on the first being true.
- Add comments above each logical group of steps in the Advanced Editor. Future readers should be able to understand the purpose of each transformation block without running it.

Common Issues and Solutions

Text.Proper capitalises unwanted abbreviations

Cause: `Text.Proper` treats every word boundary as a capitalisation point: 'USA' becomes 'Usa', 'LLC' becomes 'Llc'.

Solution: Apply `Text.Proper` first, then use `Text.Replace` to restore specific abbreviations:
`Text.Replace(Text.Proper([Name]), "Llc", "LLC")`.

Number.Round returns slightly unexpected results

Cause: Floating-point numbers cannot always be represented exactly in binary, leading to results like 2.549999... being rounded to 2.54 instead of 2.55.

Solution: Use `Currency.Type` for monetary values — it uses 128-bit fixed-point arithmetic that avoids floating-point rounding issues.

Conditional column crashes on null values

Cause: A comparison like `[Sales] > 1000` where `[Sales]` is null returns null (not false), which can propagate unexpectedly through calculations.

Solution: Add a null check as the first branch: `if [Sales] = null then defaultValue else if [Sales] > 1000 then...` Use `??` for simpler cases.

Text.Split produces more parts than expected

Cause: If the source string has consecutive delimiters (e.g. 'East,,West') or a trailing delimiter ('East,West,'), `Text.Split` produces empty strings in the list.

Solution: Filter the result with `List.Select(Text.Split(text, delimiter), each _ <> "")` to remove empty parts.

each _ shadowing in nested contexts

Cause: Writing `Table.AddColumn(t, "X", each List.Select(list, each _ > [Threshold]))` causes the inner `each` to shadow the outer `each` — `[Threshold]` is inaccessible in the inner context.

Solution: Use explicit parameter names for the outer function: `(row) => List.Select(list, each _ > row[Threshold])`.

Key Functions — Module 2 Quick Reference

The following table provides a concise reference for all functions introduced in Module 2. Keep this page bookmarked during lab exercises and exam preparation.

Function	Category	Syntax (simplified)	Key Behaviour
Text.Upper	Case	Text.Upper(text, optional culture)	All characters to uppercase
Text.Lower	Case	Text.Lower(text, optional culture)	All characters to lowercase
Text.Proper	Case	Text.Proper(text, optional culture)	First letter of each word capitalised
Text.Trim	Space	Text.Trim(text, optional char)	Remove leading and trailing whitespace/char
Text.TrimStart	Space	Text.TrimStart(text, optional char)	Remove leading whitespace/char only
Text.TrimEnd	Space	Text.TrimEnd(text, optional char)	Remove trailing whitespace/char only
Text.PadStart	Pad	Text.PadStart(text, count, optional char)	Left-pad to total length count
Text.PadEnd	Pad	Text.PadEnd(text, count, optional char)	Right-pad to total length count
Text.Length	Inspect	Text.Length(text)	Number of characters in text
Text.Contains	Inspect	Text.Contains(text, sub, optional comparer)	True if sub found in text
Text.StartsWith	Inspect	Text.StartsWith(text, sub, optional comparer)	True if text begins with sub
Text.EndsWith	Inspect	Text.EndsWith(text, sub, optional comparer)	True if text ends with sub
Text.Start	Extract	Text.Start(text, count)	First count characters
Text.End	Extract	Text.End(text, count)	Last count characters
Text.Middle	Extract	Text.Middle(text, start, optional count)	Substring from start position
Text.Range	Extract	Text.Range(text, offset, optional count)	Equivalent to Text.Middle
Text.PositionOf	Search	Text.PositionOf(text, sub, optional occurrence)	Zero-based index; -1 if not found
Text.PositionOfAny	Search	Text.PositionOfAny(text, chars, optional occ)	Position of first char from list
Text.Replace	Modify	Text.Replace(text, old, new)	Replace all occurrences of old with new
Text.Remove	Modify	Text.Remove(text, removeChars)	Remove all specified characters

Text.Select	Modify	Text.Select(text, selectChars)	Keep only specified characters
Text.ReplaceRange	Modify	Text.ReplaceRange(text, offset, count, new)	Replace count chars at offset
Text.Insert	Modify	Text.Insert(text, offset, newText)	Insert newText at offset
Text.Repeat	Modify	Text.Repeat(text, count)	Repeat text count times
Text.Split	Split	Text.Split(text, separator)	Split to list at separator
Text.SplitAny	Split	Text.SplitAny(text, separators)	Split at any char in separators
Text.Combine	Combine	Text.Combine(list, optional separator)	Join list of text with separator
Text.From	Convert	Text.From(value, optional culture)	Any value to text
Text.Format	Convert	Text.Format(format, arguments)	Format string with #{0} placeholders
Number.Round	Round	Number.Round(n, optional digits, optional mode)	Round to digits decimal places
Number.RoundUp	Round	Number.RoundUp(n, optional digits)	Always round away from zero
Number.RoundDown	Round	Number.RoundDown(n, optional digits)	Always round toward zero
Number.RoundTowardZero	Round	Number.RoundTowardZero(n, optional digits)	Truncate toward zero
Number.Abs	Arith	Number.Abs(n)	Absolute value
Number.Sign	Arith	Number.Sign(n)	1, -1, or 0
Number.Mod	Arith	Number.Mod(n, divisor)	Remainder after division
Number.IntegerDivide	Arith	Number.IntegerDivide(n, divisor)	Integer quotient (floor division)
Number.Power	Math	Number.Power(base, power)	base raised to power
Number.Sqrt	Math	Number.Sqrt(n)	Square root
Number.Exp	Math	Number.Exp(n)	e raised to n
Number.Log	Math	Number.Log(n, optional base)	Log base 'base' of n (default: ln)
Number.IsNaN	Check	Number.IsNaN(n)	True if NaN
Number.IsEven	Check	Number.IsEven(n)	True if even integer
Number.IsOdd	Check	Number.IsOdd(n)	True if odd integer
Number.From	Convert	Number.From(value, optional culture)	Any value to number
Number.ToText	Convert	Number.ToText(n, optional format)	Number to formatted text

TIP The complete formula reference covering all M functions across all 12 modules is available in the companion document: [PowerQuery_M_Formula_Reference.pdf](#)

Module Summary

- M provides a rich set of Text. functions covering case conversion, whitespace handling, padding, length measurement, substring extraction, searching, modification, splitting, combining, and conversion. All are case-sensitive by default; pass a Comparer for case-insensitive operations.
- M supports several numeric types: type number (64-bit double), Int64.Type (integers), and Currency.Type (fixed-point, 4 decimal places). Choose Currency.Type for monetary columns.
- Four rounding functions cover all scenarios: Number.Round (standard), Number.RoundUp (ceiling), Number.RoundDown (floor), Number.RoundTowardZero (truncate). Number.Round supports banker's rounding via RoundingMode.ToEven.
- M's if-then-else is an expression, not a statement. The else branch is mandatory. Nest conditions in the else branch for multi-branch logic.
- The and and or operators do NOT short-circuit. Use nested if expressions when the second condition depends on the first being safe to evaluate.
- Null comparisons return null (not false). Always handle nulls explicitly in conditions using = null checks or the ?? null-coalescing operator.
- The each keyword is shorthand for a single-parameter anonymous function (_) => expression. Inside table contexts, _ is the current row; inside list contexts, _ is the current element.
- Use Table.TransformColumns for in-place column modification and Table.AddColumn for adding new derived columns while preserving originals.

Next Module Module 3: Working with Dates, Times, and Durations — Covers all Date.*, Time.*, DateTime.*, and Duration.* functions; building a complete Date dimension table; time intelligence patterns; and working with time zones in M.